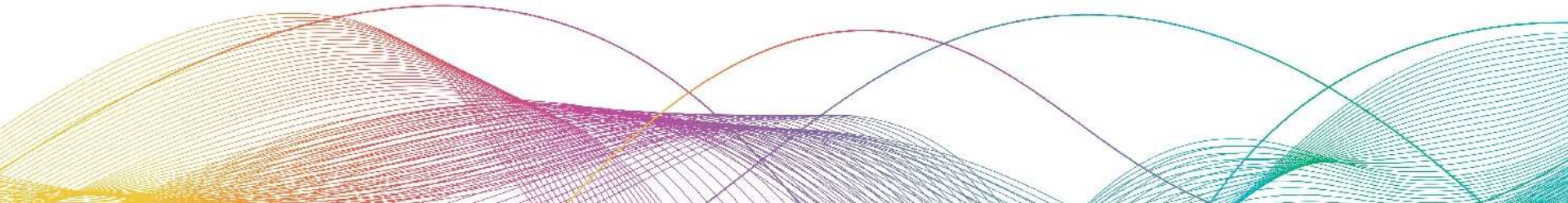
05
Build A Complex Application

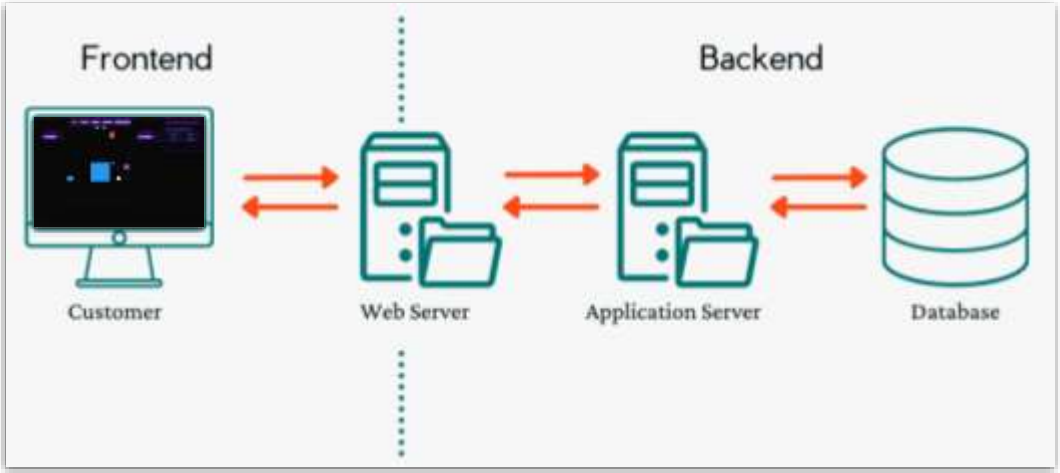


Introduction to Web Applications

A web application is made up of two main parts:

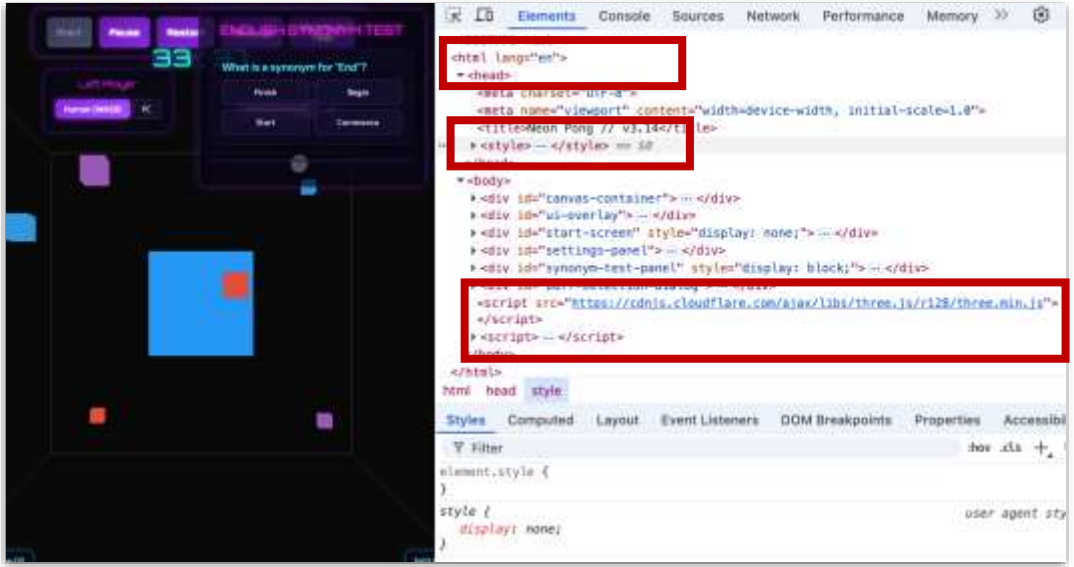
- **Frontend**: what users see and interact with
- **Backend**: the server that processes data and logic

Together, they create dynamic, interactive websites



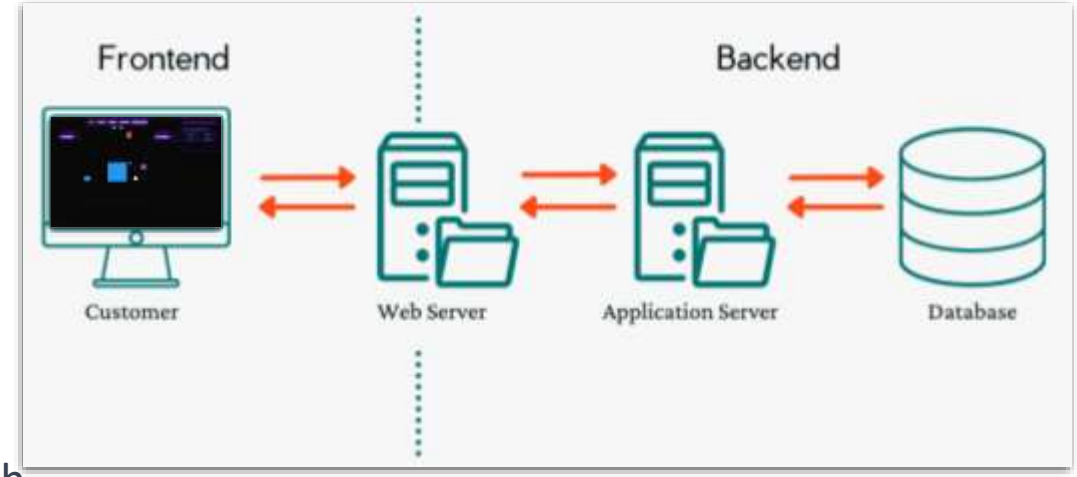
The Frontend

- Runs in the **browser**
- Built with:
  - HTML → structure
  - CSS → style
  - JavaScript → behavior
- Example: displaying a quiz question to the user



The Backend

- Runs on the **server**
- Handles:
  - Data storage (databases)
  - Logic and computation
  - Security and validation
- Communicates with frontend through HTTP requests
  - JSON or HTML responses



When Do We Need a Backend?

- When we need to **store data** or **process user input**
- When we need **secure checks** or **business logic**
- Example :
  - A test question appears in the frontend
  - The user submits an answer
  - The backend checks if the answer is correct and returns a result

1. Frontend shows a question, e.g. “What is 2 + 2?”
2. User enters an answer, e.g. “5” and clicks “Submit”
3. Frontend sends “{answer: 5}” to backend
4. Backend returns result → “Incorrect!”
5. Frontend displays the result

Popular Python Backend Frameworks

- **Flask**
  - Lightweight and flexible
  - Great for small to medium apps
- **Django**
  - Full-featured and opinionated
  - Includes admin, authentication

Both can serve HTML templates and static files

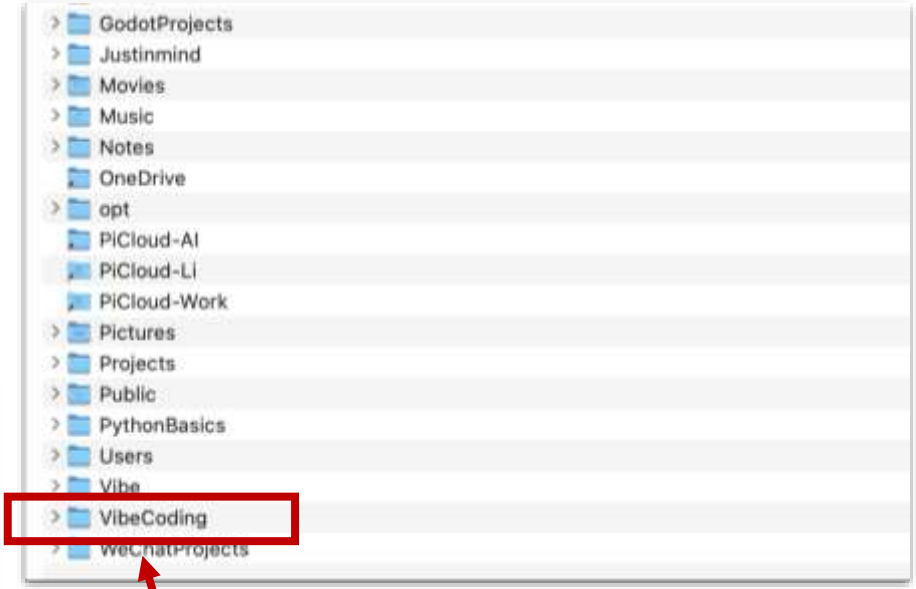
Preparation for Flask

- Open the command line window
  - macOS: Terminal
  - Windows: Anaconda Prompt

cd /Users/[name]

cd C:\Users\[name]

```
mkdir VibeCoding
cd VibeCoding

pip install flask
```



You should see the created folder in the home directory.

How Flask Works

- Flask maps **URLs** to **Python functions**
- These functions return **HTML** or **JSON** responses

Open Terminal (Terminal for macOS, Anaconda Prompt for Windows)

```
python hello.py
```

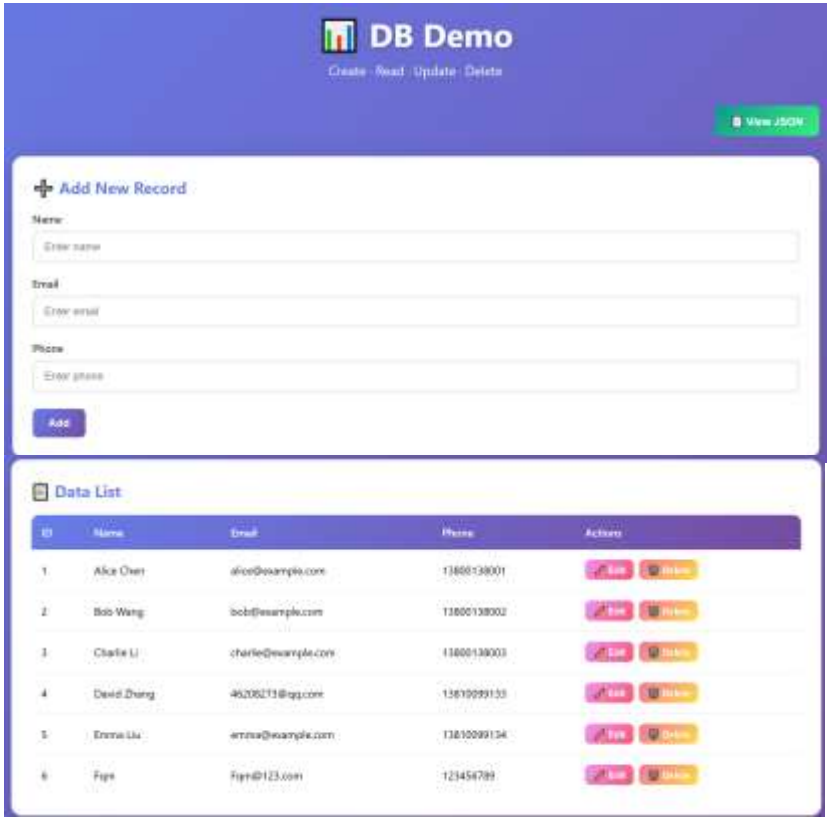
Open “localhost:6660” in browser

```
# hello.py

from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Hello, World!"

if __name__ == '__main__' :
    app.run(debug=True, port=6660)
```

DB-Demo

Prompt:

1. A simple web application demonstrating database CRUD (Create, Read, Update, Delete) operations
2. HTML+Flask+DB （JSON）
3. The interface is nice.



Plane-Shooter

Using HTML+Flask+DB (JSON) technology stack, create a 2D vertical shooting game or other games or learning applications.

Requirements:

1. Other students can access it through the internet and start using it by entering their names.
2. During use, there are points and a ranking list, sorted according to the high or low points.

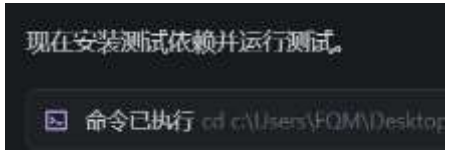
Other students can access it by entering this address in their browser

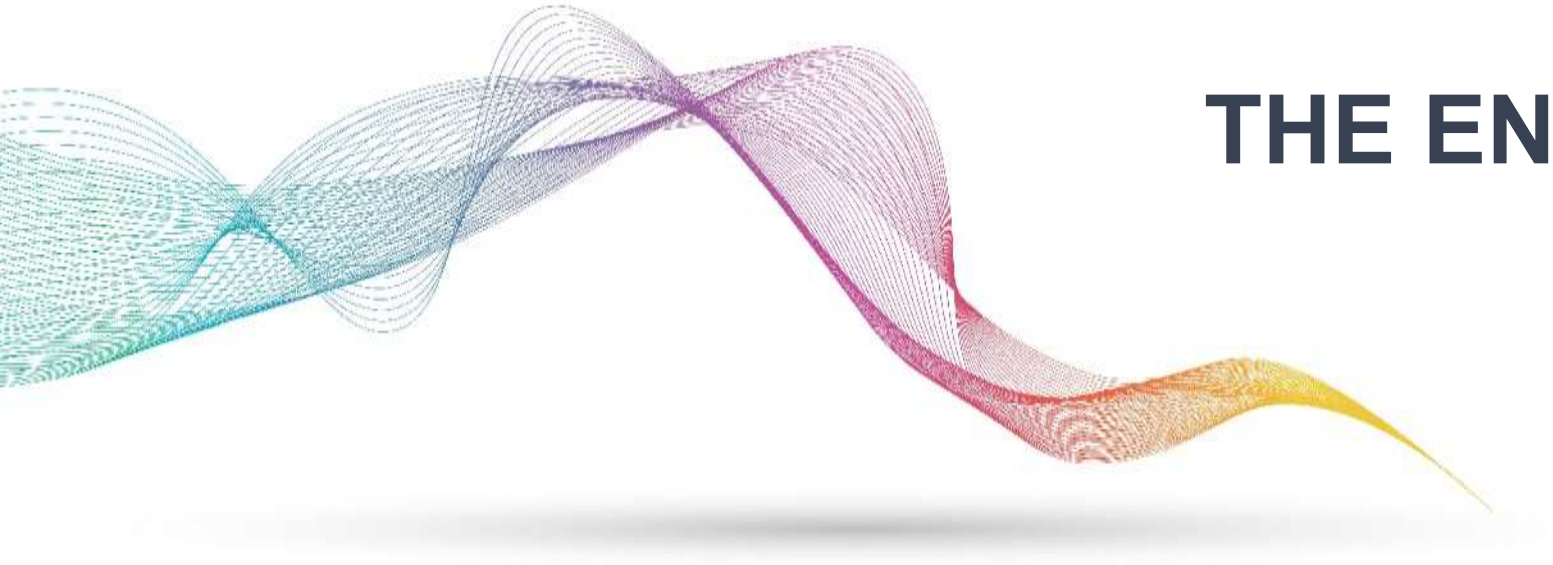
Plane-Shooter

Automated Testing

Steps:

1. Write a test checklist using AI.
2. Use AI to install testing environment dependencies (When first used).
3. Create test file structure using AI.
4. Conduct API testing
5. Conduct UI testing
6. Run all tests (some tests require manual completion)
7. View test results





THE END